

cloud-architecture-lab

Claude Code Tooling Reference — how `.claude/`, `docs/standards/`, and the daily workflow fit together
Generated 2026-06-16 · current state: Day 009 complete, Day 010 next

Contents

- 1. System map — the four layers
- 2. Inside `.claude/` — orchestration layer
 - 2.1 Slash commands
 - 2.2 Skills
 - 2.3 Hooks & settings
 - 2.4 Instructions (the master workflow)
- 3. `docs/standards/` — the knowledge library
- 4. The daily workflow, step by step
- 5. A day in the life — worked example
- 6. `scripts/` — automated guards
- 7. Findings from this audit
- 8. Expected results

1. System map — the four layers

This repo is two things at once: a .NET 8 AI gateway ([src/lab-observability-api/](#)) and a self-documenting **process** for building it day by day, encoded almost entirely as Claude Code configuration. The process layer has four tiers, each one consuming the layer below it.

1. Instructions

`.claude/instructions/`

The master script.
Defines STEP 0–12 and
what model/surface
runs each one.

2. Commands & Skills

`.claude/commands/`
`.claude/skills/`

The verbs the
instructions invoke. A
command is a one-shot
recipe; a skill is reusable
know-how a command
or you can pull in.

3. Standards

`docs/standards/`

The knowledge
commands/skills read
from and write
decisions into. Naming
rules, error taxonomy,
cert mapping, etc.

4. Artifacts

`docs/notes/` ,
`docs/adr/` , `src/`

What actually gets
produced: code, ADRs,
day notes, the
changelog.

Two enforcement mechanisms run underneath all four layers:

Hooks

`.claude/settings.json`

Fire automatically on every `Write / Edit` — no
command has to remember to call them.

Scripts

`scripts/*.js`

Stateless checkers (`md-backtick-wrap.js` , `symbol-`

`drift-check.js`) invoked by hooks, commands, and STEP 8 of the workflow.

2. Inside `.claude/` — orchestration layer

2.1 Slash commands (`.claude/commands/*.md`)

Each file is a prompt template Claude Code expands when you type the matching `/name` . There are 7 registered commands.

Command	Workflow step	What it does	Reads	Writes
<code>/new-day N slug</code>	STEP 3	Scaffolds the empty day skeleton — folder, summary/checklist stubs, architecture doc, Infra template — before any content exists	<code>docs/notes/_index.md</code>	New <code>Day-NNN/</code> folder, <code>changelog.md</code> section, architecture stub, Infra stub
<code>/adr title</code>	STEP 6	Creates the next-numbered ADR from the standard template, Status: Proposed	<code>adr-writer/SKILL.md</code> , existing ADR numbers	New <code>docs/adr/ADR-NNN-*.md</code>
<code>/collab-lens N</code>	STEP 6	Inserts a bounded "who does this serve" block into <code>01-summary.md</code> , phase-filtered against the collaboration map	<code>collaboration-lens/SKILL.md</code> , <code>collaboration-map.md</code> , <code>01-summary.md</code>	<code>01-summary.md</code> (insert), <code>changelog.md</code> (row)
<code>/deploy</code>	STEP 9	Pre-deploy gates (tests + pillars audit) → Kudu zip publish → post-deploy verification	<code>azure-deploy/SKILL.md</code> , <code>Infra/Day-NNN/appsettings-template.md</code>	Azure App Service, <code>06-deployment-log.md</code>
<code>/cert-update N [all]</code>	STEP 10	Reads the day's Certification Reinforcement section, generates concepts/practice-q/resources for one touched domain (or all, with <code>all</code>)	<code>01-summary.md</code> , MS Learn MCP	<code>docs/certifications/<exam>/domains/.../*.md</code>
<code>/cert-scaffold EXAM</code>	one-time per exam	Fetches the official skills outline, builds the domain folder structure — does not generate explanations	MS Learn certification page	<code>docs/certifications/<EXAM>/</code> structure
<code>/repo-audit N</code>	STEP 12	9-check end-of-day health scan: folder completeness, ADR structure, lint, backticks, <code>CLAUDE.md</code>	Everything	Auto-fixes lint/backticks; reports the rest

accuracy,
changelog
coverage, cert
coverage, provider
abstraction,
symbol drift

Not a real command: `/devil`, `/pitch`, and `/10x` are referenced in `daily-workflow.md` 's command table but are *not* files in `.claude/commands/`. They are "paste this exactly" prompt blocks embedded directly in `daily-workflow-steps.md` (STEP 4, 5, and 11) — reasoning gates you run inline in a chat, not slash commands the harness resolves. Active from Day 010+.

2.2 Skills (`.claude/skills/*/SKILL.md`)

Skills are reusable knowledge packs. Unlike commands, they're not typed directly — they're invoked by name from inside a command's instructions, or pulled in by Claude Code automatically when the situation matches their description.

Skill	Invoked by	What it knows
<code>adr-writer</code>	<code>/adr</code>	ADR template, heading structure, "verb-led title", at least 2 alternatives with rejection reasons
<code>azure-deploy</code>	<code>/deploy</code>	The Kudu zip path and why <code>az webapp deploy</code> fails on this network; SSL workarounds; post-deploy verification checklist
<code>collaboration-lens</code>	<code>/collab-lens</code>	How to pick 1 primary + ≤ 2 secondary collaborators from <code>collaboration-map.md</code> , phase-filtered, and format the 4-level compression block
<code>dotnet-api-conventions</code>	auto (writing controllers/services in <code>src/</code>)	Provider abstraction rules, options pattern, the <i>real</i> error-handling pipeline (inline <code>app.Use</code> , not a middleware class)
<code>observability-net</code>	auto (logging/tracing work)	Serilog + OpenTelemetry wiring, correlation IDs, the <code>TryExtractUsage</code> 4-tuple cache-format gotcha
<code>pillars-audit</code>	<code>/deploy</code> (gate b), STEP 8/12	The 6-pillar pre-deploy audit (5 WAF pillars + Responsible AI), 34 codebase-specific GREEN/YELLOW/RED checks

2.3 Hooks & settings

File	Role	Behavior
<code>.claude/settings.json</code>	Project-level config — this is the closest thing to "appsettings.json at the .claude level" (there is no literal file by that name; this is its functional equivalent for the harness itself, as opposed to <code>src/lab-observability-api/appsettings.json</code> which configures the .NET app)	Registers one <code>PostToolUse</code> hook: after any <code>Write / Edit</code> on a <code>.md</code> file, run <code>markdownlint --fix</code> on it automatically. Also holds the permission allow/deny lists (denies reading secrets/ <code>.pfx</code> / <code>appsettings.Production.json</code> ; denies destructive git/az commands) and enables the Microsoft Docs MCP plugin.
<code>.claude/settings.local.json</code>	Machine-local permission overrides (not committed for secrets, but this one just allow-lists a few read-only Azure/PowerShell commands)	Merges on top of <code>settings.json</code>
<code>.claude/hooks/cert-tag.json</code>	A second hook definition, separate from <code>settings.json</code>	On every file touch, regex-matches the path against topic keywords (Telemetry, Resilience, Provider, Bicep, Cache, ...) and appends a cert-domain tag to <code>.claude/cert-tags-today.txt</code>

Finding: nothing in this repo currently reads `.claude/cert-tags-today.txt` — `/cert-update` derives touched domains purely from `01-summary.md`'s Certification Reinforcement section. The cert-tag hook's output is generated but unconsumed today.

2.4 Instructions — the master workflow

`daily-workflow.md` is the one-page overview (the STEP 0–12 table, the four rules, the auto-print handoff mechanic). `daily-workflow-steps.md` is the long version — the literal prompt text for every step, including the `/devil` / `/pitch` / `/10x` gate scripts. You read the overview to orient; you paste from the steps file to actually run a step.

3. docs/standards/ — the knowledge library

26 files. Each is read on-demand (none are in permanent hot context except `_principles.md`, `naming-conventions.md`, `azure-environment.md` per `CLAUDE.md`'s context-discipline rules). Grouped by what they govern:

Governs	Files
Posture & career	<code>_principles.md</code> , <code>career-path.md</code> , <code>competitive-intelligence.md</code> , <code>architect-thinking-template.md</code> , <code>graveyard.md</code>
Naming & environment	<code>naming-conventions.md</code> , <code>azure-environment.md</code>
Build conventions	<code>testing-standard.md</code> , <code>commit-convention.md</code> , <code>error-handling-standard.md</code> , <code>api-versioning.md</code> , <code>dependency-policy.md</code>
Cost & performance	<code>cost-governance.md</code> , <code>slo-performance.md</code> , <code>multi-turn-context.md</code>
Security & compliance	<code>security-standard.md</code> , <code>responsible-ai.md</code>
Operations	<code>monitoring-runbook.md</code> , <code>kql-cookbook.md</code> , <code>provider-onboarding.md</code>
Evaluation & future paradigms	<code>eval-framework.md</code> , <code>agent-patterns.md</code> , <code>rag-patterns.md</code>
People & communication	<code>collaboration-map.md</code> , <code>public-documentation.md</code> , <code>portfolio-strategy.md</code>

4. The daily workflow, step by step

Step	Surface	Model	What happens
0 CLOSE (prev)	Claude Code	Sonnet	Verify the previous day is fully frozen — audit, posture-check, changelog row all present — before starting anything new
1 PRUNE	Manual	—	Move the previous day's hot-context files out; rotate which ADRs stay loaded
2 DECIDE	Chat (new)	Sonnet	Lock the day's workload and slug
3 SCAFFOLD	Claude Code	Sonnet	<code>/new-day N slug</code> — build the empty skeleton
4 DRAFT	Chat (new)	Sonnet	Full 13-section Day N summary; <code>/pitch</code> gate validates the CEO framing
5 ADR	Chat (new)	latest Opus	Reason out the architectural decision (skip if none this day); <code>/devil</code> gate stress-tests it
6 POPULATE	Claude Code	Sonnet	Fill summary + checklist, run <code>/adr</code> , run <code>/collab-lens</code>
7 BUILD	Claude Code	Sonnet	Implement each phase of the day's plan
8 TEST + AUDIT	Claude Code	Sonnet	<code>dotnet test</code> , the 6-pillar audit, <code>symbol-drift-check.js</code> — all before deploy is allowed
9 DEPLOY	Claude Code	Sonnet	<code>/deploy</code> + post-deploy verification
10 DOCUMENT	Claude Code	Sonnet	Update docs, commit, <code>/cert-update</code>
11 REFLECT	Chat (new)	latest Opus	Posture check (5 questions); <code>/10x</code> gate audits the scale ceiling (mandatory Day 051+)
12 CLOSE	Claude Code	Sonnet	Save posture check, final commit, <code>/repo-audit</code>

Rule of thumb: Sonnet does all the building; the latest Opus is reserved for the two steps where reasoning depth — not execution — is the actual deliverable (ADR reasoning, posture reflection).

5. A day in the life — worked example

Tracing one real artifact, `IBatchChatModelProvider`, through the whole stack on the day it was built (Day 008):

STEP 2 (chat) — decide: "batch API for cost control" is the Day 008 workload

STEP 3 — `/new-day 8 batch-api-cost-controls` creates `docs/notes/Day-008/`, a `## Day 008` section in `changelog.md`, and `docs/architecture/day-008-batch-api-cost-controls.md`

STEP 5 (chat, latest Opus) — reasons out: new seam vs. extending `IChatModelProvider` → new seam wins (different computation model, not just different speed)

STEP 6 — `/adr introduce-parallel-batch-provider-abstraction` creates `ADR-010`, Status: Proposed

STEP 7 — builds `IBatchChatModelProvider.cs`, `ClaudeBatchChatModelProvider.cs`, `AiBatchController.cs`; the `dotnet-api-conventions` skill governs the shape

STEP 8 — `pillars-audit` finds two RED items (missing `MaxPromptLength` guard) → fixed → re-audit GREEN; `symbol-drift-check.js` confirms every new type is referenced correctly everywhere it's mentioned

STEP 9 — `/deploy`; the `azure-deploy` skill's Kudu path ships it to `app-ai-lab-api-dev-eastus-gio`

STEP 10 — `/cert-update 008` writes AI-102 and AZ-305 domain content citing the batch pattern; `changelog.md`'s Day 008 section gets every file logged

STEP 12 — `/repo-audit 8` confirms ADR-010 has all required headings, no drift, day folder complete

6. scripts/ — automated guards

Script	Triggered by	Checks
<code>md-backtick-wrap.js</code>	<code>settings.json</code> hook (every <code>.md</code> write/edit), <code>/repo-audit</code> Check 4	Wraps bare file paths, slash commands, and HTTP paths in backticks per the markdown convention. Idempotent — re-running on clean content changes nothing.
<code>symbol-drift-check.js</code>	STEP 8, <code>/repo-audit</code> Check 9	Rebuilds the actual symbol/telemetry-name table from <code>src/</code> on every run; flags any backtick-wrapped doc reference that doesn't match current code — including symbols that were <i>documented but never built</i> . Also flags single-letter backtick spans as likely truncation/corruption (added 2026-06-16 after recovering 80 such occurrences).

7. Findings from this audit

Fixed — severe: a buggy `md-backtick-wrap.js` replace callback (before its fix in commit `1e33390`) had collapsed full paths/commands down to single letters inside backticks across 50 files, 80 occurrences — `/health` → ``h``, `/cert-update`

→ ``c``, etc. Recovered every instance from git history and added a permanent guard so this class of corruption can't recur silently.

Fixed: 6 source files (`ApiError.cs` , `TestController.cs` , `HttpContextExtensions.cs` , `CorrelationIdMiddleware.cs` , `ClaudeProviderException.cs` , the main `.csproj`) existed with zero changelog provenance — every `src/` file now traces to a Day NNN.

Open, not a bug: `.claude/hooks/cert-tag.json` generates tags nothing consumes (see §2.3). Worth deciding whether to wire it into `/cert-update` or remove it.

Open, accepted debt: ADR-001/002/003 predate the "Alternatives Considered" convention. Not fixed — adding that section retroactively to an Accepted ADR isn't a "factual correction" under this repo's own ADR-immutability rule.

8. Expected results

When every layer is working together correctly, a closed day means: the code builds and tests pass; the ADR (if any) has all required headings and no open placeholders; every touched file has a `changelog.md` row under that day's section; every cert domain marked Primary/Secondary in the day's summary has a corresponding day-mapping row; `symbol-drift-check.js` and `markdownlint` both report zero findings; the deploy succeeded and was verified against the live App Service; and the posture-check honestly names what's still YELLOW. `/repo-audit` is the single command that verifies all of this at once — a clean run is the actual definition of "Day N is done."